

Chapter 6

Synchronization Tools

Chih-Yu Lin

National Taiwan Ocean University

Updated date: 2025/7/16

Review

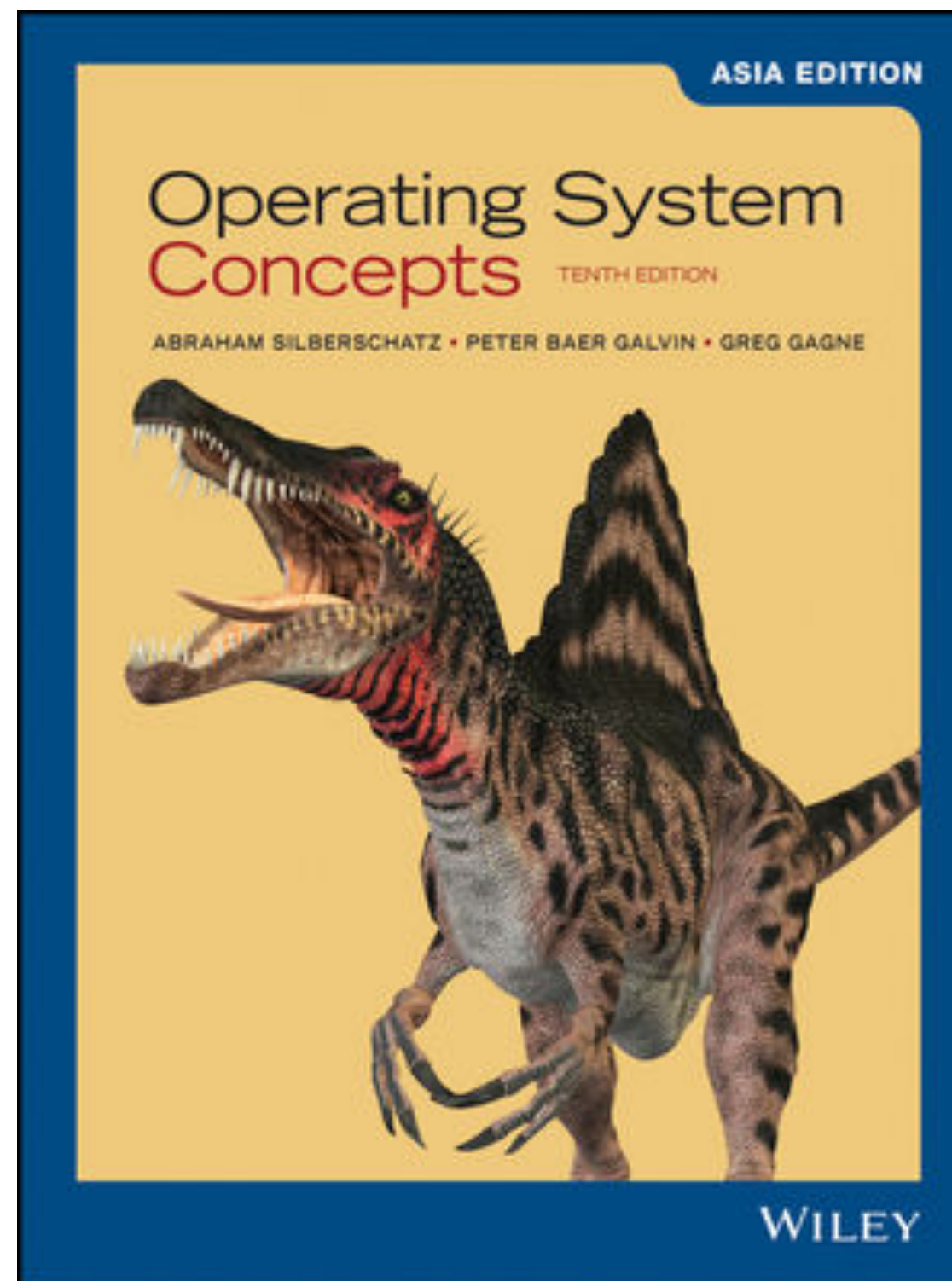
- Processes / Threads

- Context Switch



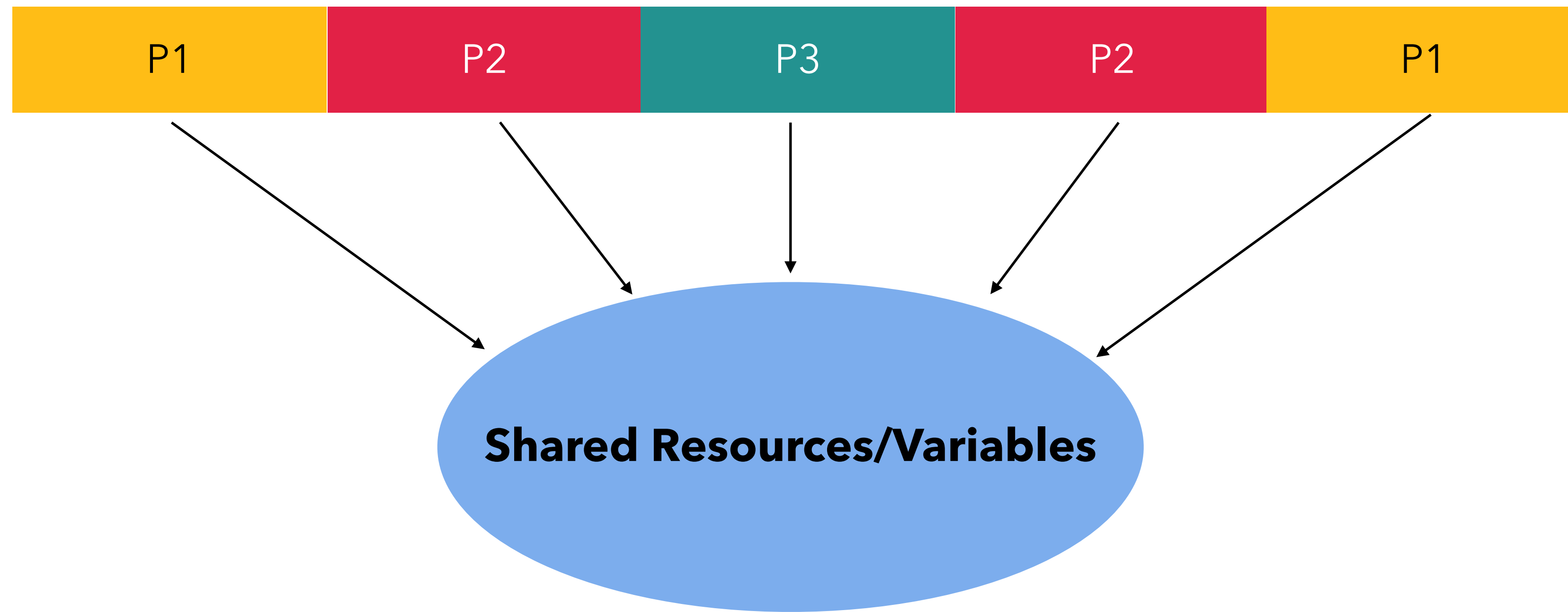
Textbook

- Abraham Silberschatz, Peter B. Galvin, Greg Gagne, "Operating System Concepts, Asia Edition," 10/e, ISBN: 978-1-119-58616-6, Wiley



Multiple **Processes** Share the Same **Resource**

多个 Process 共同使用一个资源, 才会有同步问题!



Outline

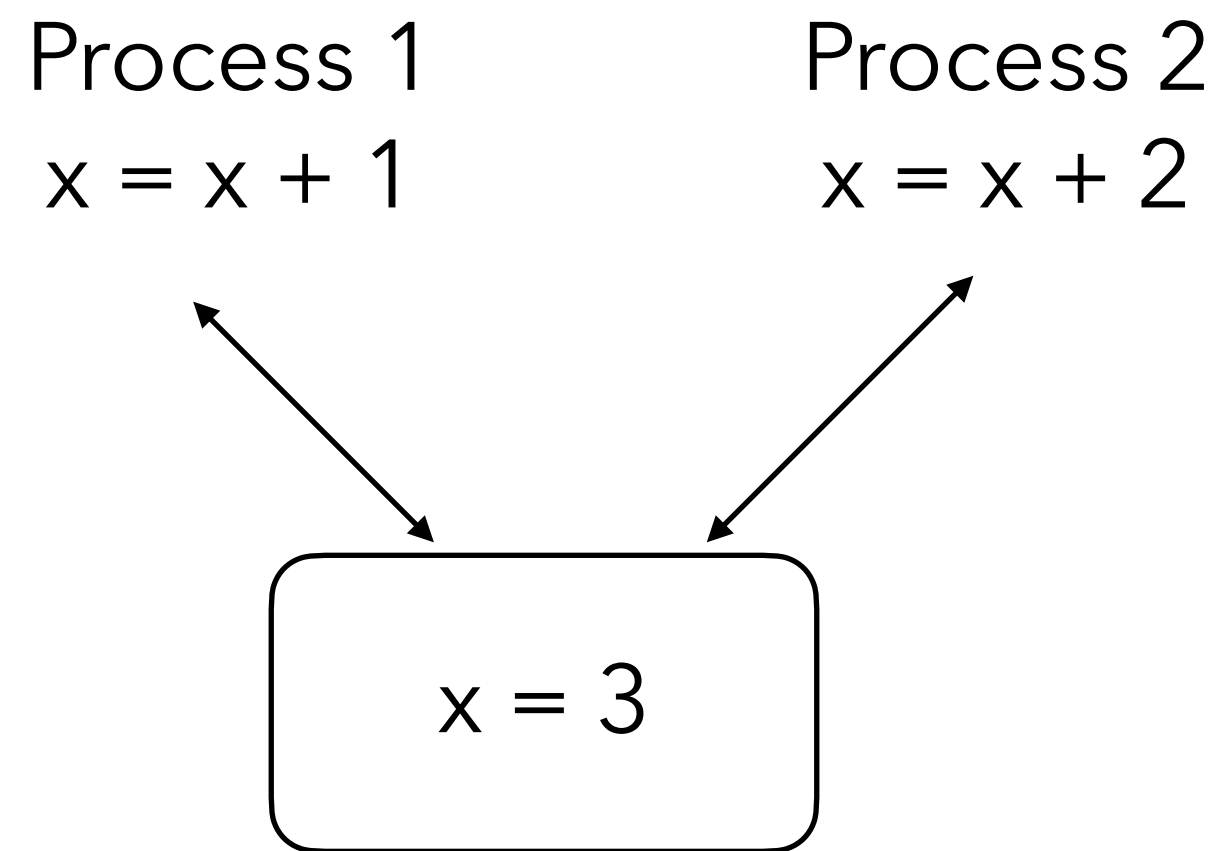
- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Outline

- Race Condition 競爭條件
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Background

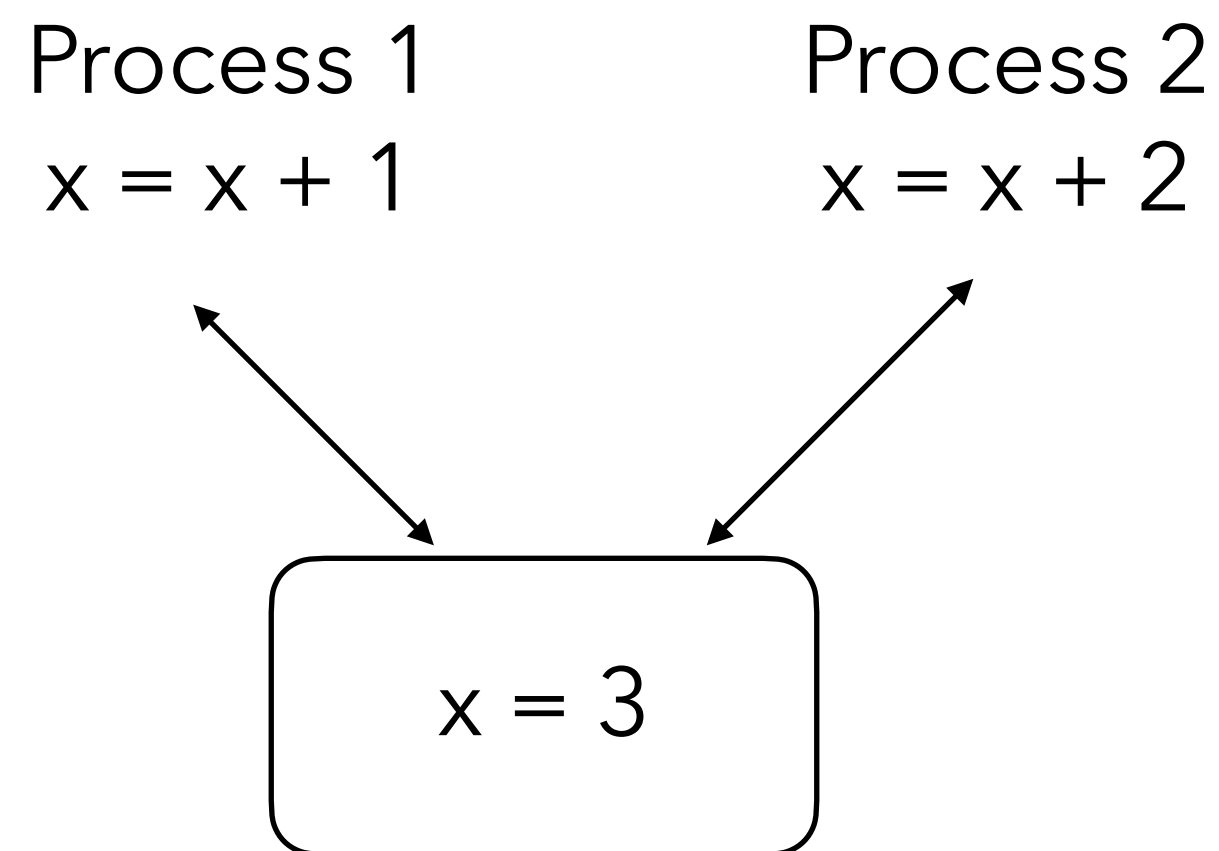
- Processes / Threads
- Context Switch
- Cooperating Processes
 - Share Data
 - Shared Memory (Section 3.5)



Background

2 Process, 共同存取一个变数

- Processes / Threads
- Context Switch
- Cooperating Processes
 - Share Data
 - Shared-Memory (Section 3.5)



↓
Expect Result:
若有作好同步,
x 结果应该要是 6

Execution Order

The result is stored in a register.

1. Process 1: Load x
2. Process 1: Add 1 $x \rightarrow 4$
3. Process 2: Load x
4. Process 2: Add 2 $x \rightarrow 5$
5. Process 2: Store x $\rightarrow$ 存 5
6. Process 1: Store x Result: 4
 $\hookrightarrow$ 存 4, Result 4!

1. Process 1: Load x
2. Process 1: Add 1 $x \rightarrow 4$
3. Process 2: Load x
4. Process 2: Add 2 $x \rightarrow 5$
5. Process 1: Store x $\rightarrow$ 存 4
6. Process 2: Store x Result: 5
 $\hookrightarrow$ 存 5, Result 5

Race Condition

競爭條件 / 競態條件
必考，會考名詞解釋

- Several processes access and manipulate the same data concurrently and the outcome of the execution depends on the particular order in which the access takes place

```
[lincyu@Chih-YudeMacBook-Pro Programs] % java RaceConditionExample.java
addValue: 2, milliseconds: 1543
addValue: 1, milliseconds: 1778
4
[lincyu@Chih-YudeMacBook-Pro Programs] % java RaceConditionExample.java
addValue: 2, milliseconds: 2655
addValue: 1, milliseconds: 382
5
[lincyu@Chih-YudeMacBook-Pro Programs] %
```

```
int counter = 0; // 共享資源
void increment() {
    for (int i = 0; i < 100000; ++i) {
        counter++; // 沒有加鎖，會產生競爭條件
    }
}

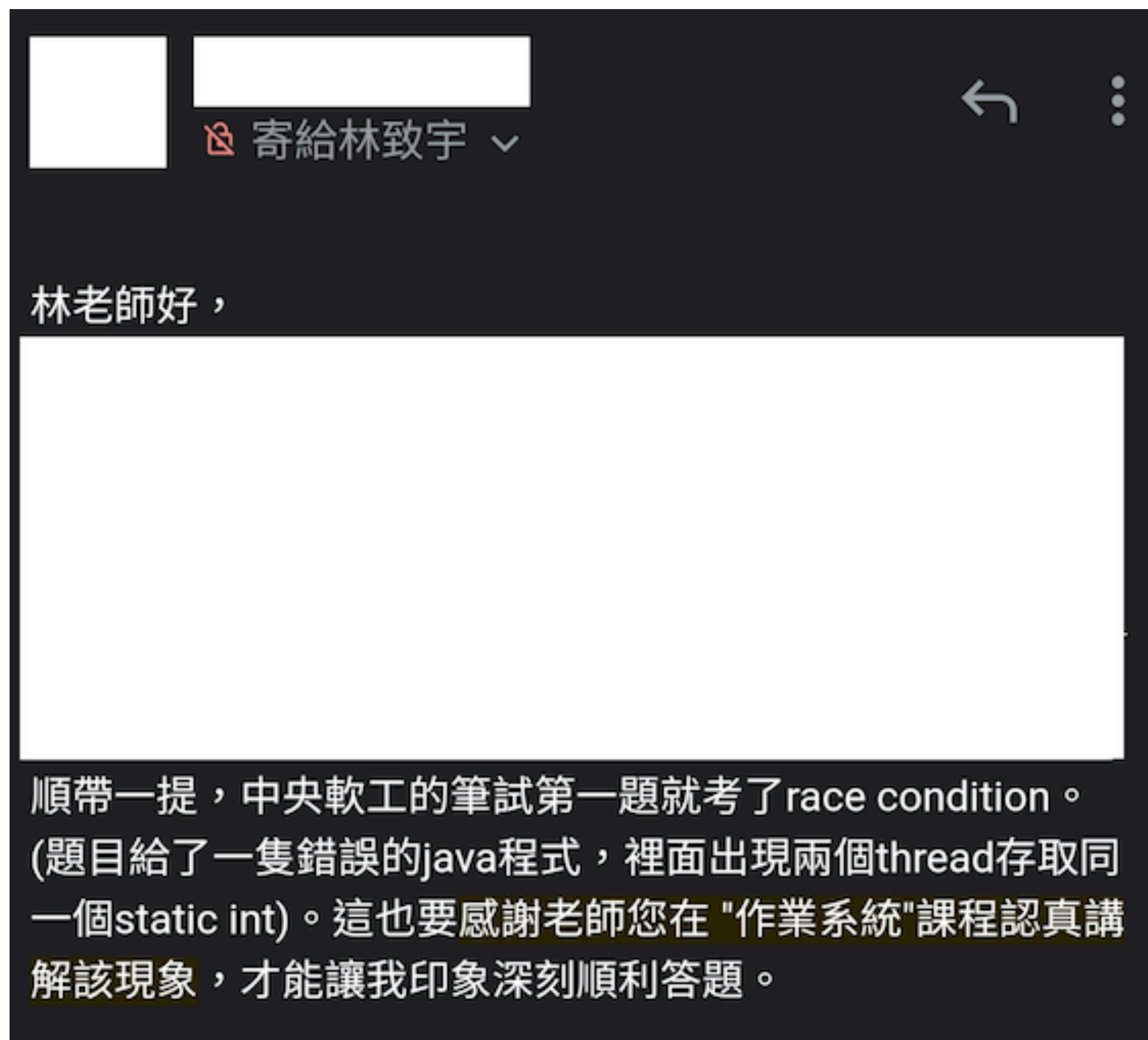
int main() {
    std::thread t1(increment);
    std::thread t2(increment);
    t1.join();
    t2.join();
    std::cout << "Final counter value: " << counter << std::endl;
    return 0;
}
```

- Synchronization

- RaceConditionExample.java

- RaceCondition.cpp

Race Condition



Producer Consumer (Sec. 3.5)

```
#define BUFFER_SIZE 10
```

```
typedef struct {
```

```
    . . .
```

```
} item;
```

```
item buffer[BUFFER_SIZE];
```

```
int in = 0;
```

```
int out = 0;
```



Producer Consumer (Sec. 3.5)

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Producer
Process

out = 0 in = 1



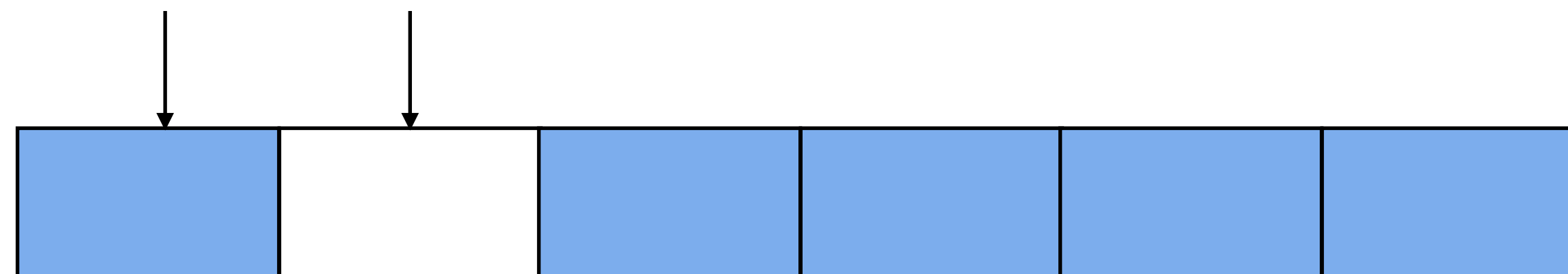
Producer Consumer (Sec. 3.5)

```
item next_produced;

while (true) {
    /* produce an item in next produced */
    while (((in + 1) % BUFFER_SIZE) == out)
        ; /* do nothing */
    buffer[in] = next_produced;
    in = (in + 1) % BUFFER_SIZE;
}
```

Producer
Process

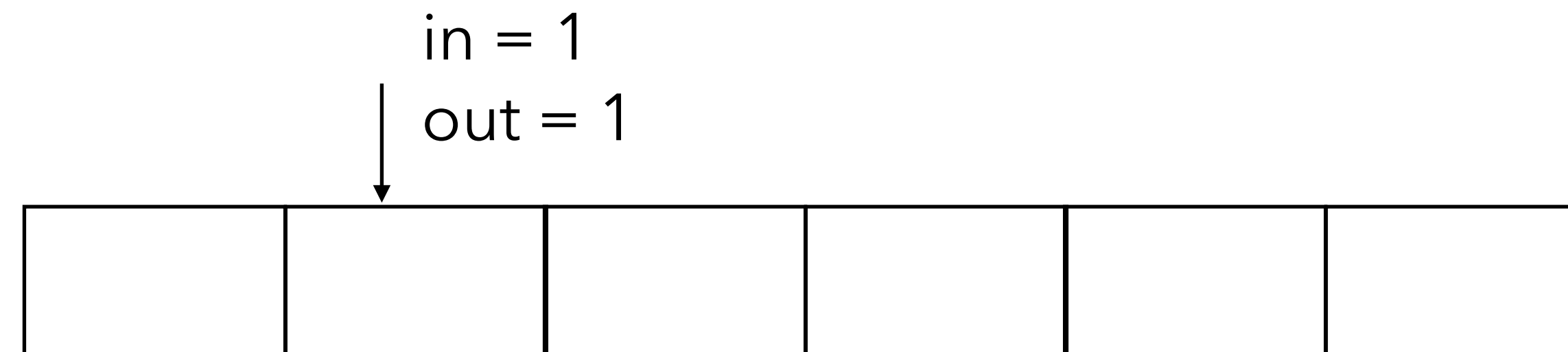
in = 0 out = 1



Producer Consumer (Sec. 3.5)

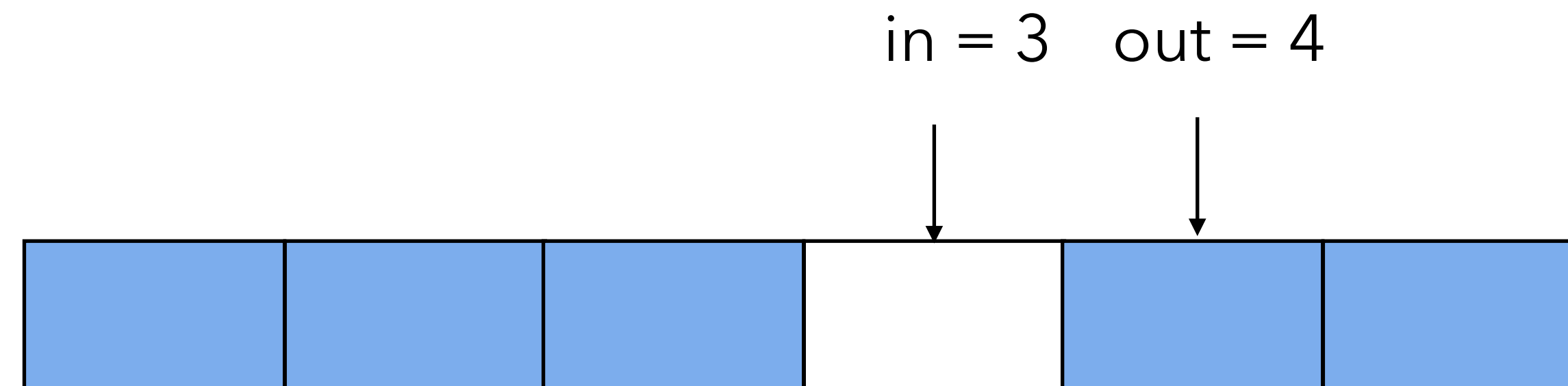
Consumer
Process

```
item next_consumed;  
  
while (true) {  
    while (in == out)  
        ; /* do nothing */  
    next_consumed = buffer[out];  
    out = (out + 1) % BUFFER_SIZE;  
  
    /* consume the item in next consumed */  
}
```



Producer Consumer (Sec. 3.5)

1. The buffer is **empty** when $in == out$
2. The buffer is **full** when $((in+1)\%BUFFER_SIZE) == out$
3. Allow at most $BUFFER_SIZE-1$ items (**deficiency**)



Add a new variable, **count**

- To remedy the deficiency (only BUFFER_SIZE-1 items), we add an integer variable, **count**

```
while (true) {  
    /* produce an item in next_produced */  
  
    while (count == BUFFER_SIZE)  
        ; /* do nothing */  
  
    buffer[in] = next_produced;  
    in = (in + 1)% BUFFER_SIZE;  
    count++;  
}
```

Producer
Process

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1)% BUFFER_SIZE;  
    count--;  
}
```

Consumer
Process

Add a new variable, **count**

```
register1 = count  
register1 = register1 + 1  
count = register1
```

Producer
Process

```
register2 = count  
register2 = register2 - 1  
count = register2
```

Consumer
Process

Add a new variable, **count**

(Producer) register1 = count

(Producer) register1 = register1 + 1

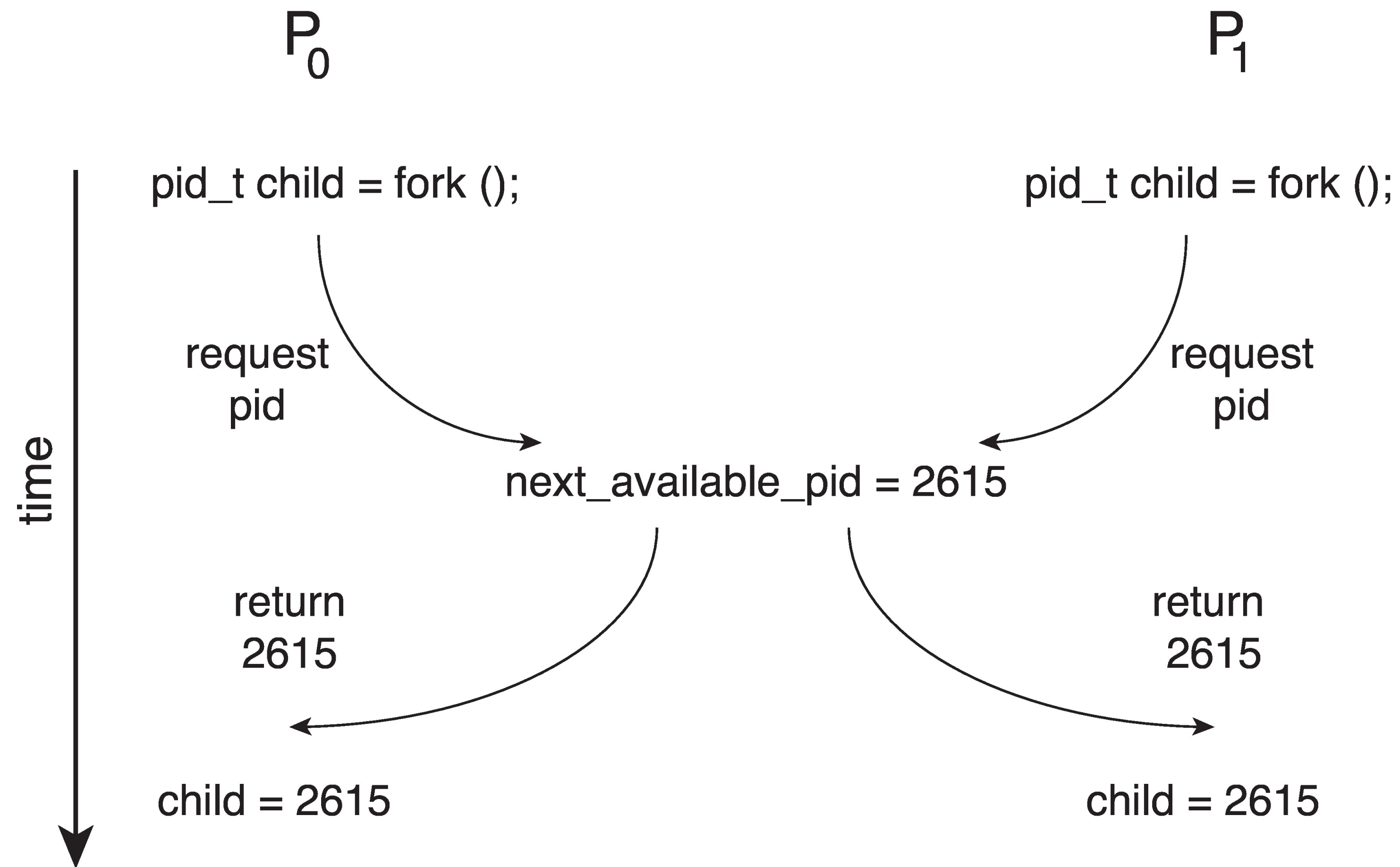
(Consumer) register2 = count

(Consumer) register2 = register2 - 1

(Producer) count = register1

(Consumer) count = register2

Another example



Outline

- Race Condition
- Critical Section 臨界區段 (為解決 Race Condition)
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Critical Section

必考，会考解釋

臨界區段、關鍵區域

(java: Synchronized)

- A multiple processes environment
- Each process has a segment of code, called a **critical section** (shared data)
- When one process is executing in its critical section, **no other process is allowed to execute in its critical section** (i.e., no two processes are executing in their critical sections at the same time)

Critical Section 就是一段程式碼，這個程式碼片段只能有一個片段執行

當有一個 Process 在其 Critical Section 執行時，其他 Process 不能執行他們自己的 Critical Section (不會有兩個 Process 同時在各自己的 C.S. 跑)

Execution Order

Process 1 enters the CS

1. Process 1: Load x

2. Process 1: Add 1

3. Process 2: Do nothing

4. Process 1: Store x

5. Process 2: Load x

6. Process 2: Add 2

7. Process 2: Store x

request permission →

entry section

critical section

exit section

remainder section

← P1 在 C.S. 中，P2 不能進自己的 C.S.

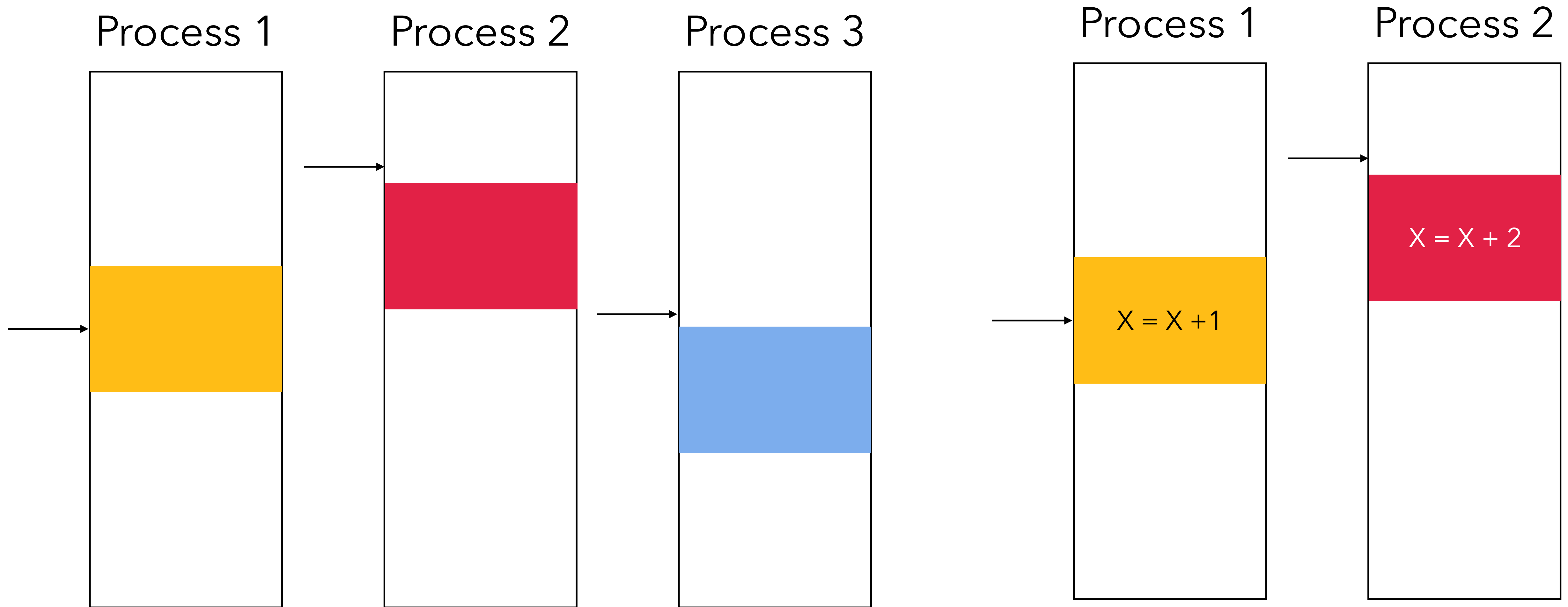
← P1 離開 C.S.

← P2 執行自己的 C.S.

} while (true);

Critical Section

* 有颜色的地方是 Critical Section



An Interrupt-based Solution

另一種解決 R.C. 的方法 - 關閉 Interrupt (不是个好方法)

- Prevent interrupts from occurring while a shared variable was being modified
 - Disable interrupts when entering the critical section
 - Enable interrupts when exiting the critical section
- Problems
 - When the codes in the critical section are needed to be run one hour?
 - A **multiprocessor** environment

The Critical Section Problem

- The critical section problem is to design a protocol that the processes can use to synchronize their activity so as to cooperatively share data

Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

→ R.C. 和
解決 R.C. 的方法 — C.S.

→ 如何把 Process 擋在 C.S. 外
怎麼建立 C.S.

Peterson's Solution

(Peterson's Algorithm)

- A classic **software-based** solution → 基於軟體來實現的(就是演算法)
- Two processes (P0, P1) share two variables
 - `int turn;`
 - 表示現在是誰要進 C.S.
 - indicate whose turn it is to enter its critical section
- `boolean flag[2]` (每個 Process 都有自己的 flag)
 - indicate if a process is ready to enter its critical section
 - 表示一 Process 是否可進入 Critical Section

Peterson's Solution

```
while (true) {
```

entry section

```
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;
```

→ This while do nothing!

```
        /* critical section */
```

exit section

```
    flag[i] = false;
```

```
        /* remainder section */
```

```
}
```

Algorithm for process P_i

Peterson's Solution

記得 turn 要設給別人!

這 2 個 while-loop
本身不做任何事!
(只負責卡住 Process)

```
while (true) {  
    flag[i] = true;   
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
    /* Critical section 是在 While 迴圈外面 */  
    /* critical section */  
    flag[i] = false;  
    /* remainder section */  
}  
  
while (true) {  
    flag[j] = true;  
    turn = i;  
    while (flag[i] && turn == i)  
        ;  
    /* critical section */  
    flag[j] = false;  
    /* remainder section */  
}
```

進 C.S. 前, 先把 flag 設為 True

用 while-loop 把 Process 卡住, 讓其不能跑到底下的 Critical Section!

離開 C.S., 把自己 flag 設為 false
讓其他 Process 可以執行 C.S.

會考

Algorithm for process Pi

Algorithm for process Pj

Peterson's Solution

- Not guaranteed to work on modern architectures
 - Instructions — 現代计算机不一定能使用(可能 turn、flag 在操作过程发生 Context Switch)
 - $\text{turn} = j, \text{flag}[i] = \text{true}$ (**single instruction rather than multiple instructions**)
- Reordering (Primary Reason) (現代 Compiler 優化的結果)

Reordering

考过!

- To improve system performance, processors and/or compilers may **reorder** read and write operations that have no dependencies

Two threads share the data:

```
boolean flag = false;  
int x = 0;
```

Thread 1 performs

```
while (!flag)  
    ;
```

```
print x
```

Thread 2 performs

```
x = 100;  
flag = true
```

Cannot be reordered

編譯器在編譯時產生
Reordering, 对调turn和flag

What is the expected output? 100

Reordering

* Peterson Solution 中, 其正確性
依賴於:
- 先表明意願: $\text{flag}[i] = \text{true}$
- 後禮讓對方: $\text{turn} = j$

期末考考過!

!!!!
这个程式是
正確的!

```
while (true) {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
  
    /* critical section */  
  
    flag[i] = false;  
  
    /* remainder section */  
}
```

Reorder

Normal

P0: $\text{flag}[0] = \text{true}$

P1: $\text{flag}[1] = \text{true}$

P1: $\text{turn} = 0$

P1: while

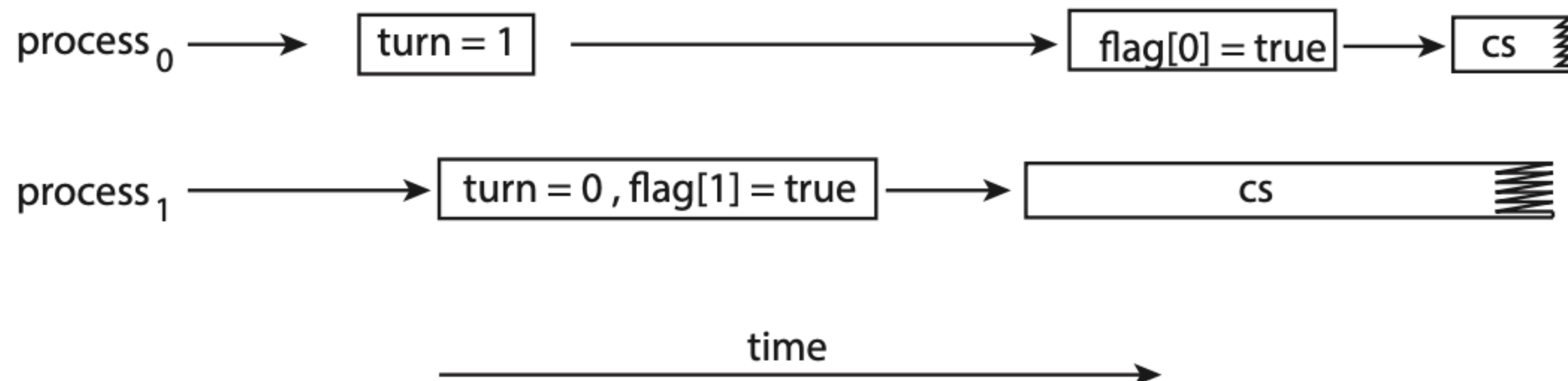
P0: $\text{turn} = 1$

P0: while

P1: CS

P1: $\text{flag}[1] = \text{false}$

P2: CS



Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Hardware Support for Synchronization

靠軟體層實現 Critical Section 不可靠 —— 靠硬體實現！

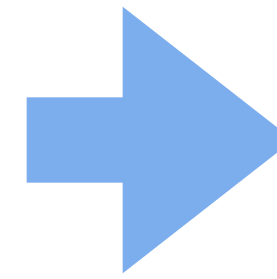
- Peterson's Solution is a **software-based** solution that involves **no special support from the operating system or specific hardware instructions**
 - Not guaranteed to work on modern computer architectures
- Three hardware instructions that provide support for solving the critical section problem
 - Memory Barriers
 - **Hardware Instructions (test_and_set(), compare_and_swap())**
 - Atomic Variables

Memory Barriers

- When a memory barrier instruction is performed, the system ensure that all load and stores are completed before any subsequent load or store operations are performed
- With respect to Peterson's solution, we could place a memory barrier between the first two assignment statements in the entry section to avoid the reordering of operations

Memory Barriers

```
while (true) {  
    flag[i] = true;  
    turn = j;  
    while (flag[j] && turn == j)  
        ;  
    /* critical section */  
    flag[i] = false;  
    /* remainder section */  
}
```



```
flag[i] = true;  
memory_barrier();  
turn = j;
```

Hardware Instructions

*是 CPU 指令!

*不可再被拆分、中途不会发生 Context Switch

- Many modern computer systems provide special hardware instructions that allow use either (1) to **test and modify the content of a word** or (2) to **swap the contents of two words** atomically
 - test_and_set()
 - compare_and_swap()

test_and_set()

- The definition of the atomic test_and_set() instruction

```
boolean test_and_set(boolean *target) {  
    boolean rv = *target;  
    *target = true;  
  
    return rv;  
}
```

給 true 就 return true
給 false 就 return false
不管傳入的 target 為何，
最後都會被改成 true

test_and_set()

兩個 Process 共同存取 lock 變數

- Mutual-exclusion implementation

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
  
    /* remainder section */  
  
} while (true);
```

```
do {  
    while (test_and_set(&lock))  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = false;  
  
    /* remainder section */  
  
} while (true);
```

std::atomic_flag

C++ 中的 test_and_set

Test_and_Set.cpp

```
std::atomic_flag lock = ATOMIC_FLAG_INIT;

int counter = 0; // 共享資源

void increment() {

    while (lock.test_and_set(std::memory_order_seq_cst)) {
        // busy-wait (spin)
    }
    // critical section
    for (int i = 0; i < 100000; ++i) {
        counter++; // 沒有加鎖，會產生競爭條件
    }
    lock.clear(std::memory_order_release); // unlock
}
```

compare_and_swap()

- The definition of the atomic compare_and_swap() instruction
 - Intel x86: cmpxchg
- https://en.cppreference.com/w/cpp/atomic/atomic_compare_exchange.html

```
int compare_and_swap(int *value, int expected, int new_value) {  
    int temp = *value;  
  
    if (*value == expected)  
        *value = new_value;  
  
    return temp;  
}
```

- 第一個整數「value」傳什麼，就 return 什麼。
- 第一個整數「value」會和第 2 個整數 expected 比較。
若相等，則將「value」改成第 3 個參數 new_value 的值。

compare_and_swap()

- 兩程式共用一變數 lock，初值為 0。
- 若只有 2 程式在 run 時，compare_and_swap 其實只用的到 0 跟 1 (同 test-and-set)，
有多個程式同時 run 時才會用到其他的數字。
- Mutual exclusion with the compare_and_swap() instruction

```
while (true) {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
}
```

```
while (true) {  
    while (compare_and_swap(&lock, 0, 1) != 0)  
        ; /* do nothing */  
  
    /* critical section */  
  
    lock = 0;  
  
    /* remainder section */  
}
```

compare_and_swap()

- Bounded-waiting mutual exclusion

期末考考过!!

```
while (true) {
```

```
    waiting[i] = true;
```

```
    key = 1;
```

```
    while (waiting[i] && key == 1)
```

```
        key = compare_and_swap(&lock, 0, 1);
```

```
    waiting[i] = false;
```

```
    /* critical section */
```

```
    j = (i+1)%n;
```

```
    while ((j != i) && !waiting[j])
```

```
        j = (j+1)%n;
```

```
    if (j == i)
```

```
        lock = 0;
```

```
    else
```

```
        waiting[j] = false;
```

```
    /* remainder section */
```

```
}
```

⊗

换成这个

```
while (waiting[i] && compare_and_swap(&lock, 0, 1))
```

即使進while 前先把 waiting[i] 設為 true

while 中仍要 check waiting 的值 → 這個檢查是有意義的!

因為有可能被其他人改掉 waiting[i] !!!

找下一個要進來
C.S. 區執行的

i=j, 找一輪了

都沒有人要進 C.S.

都沒有人要進

C.S., 把鎖打開

!!!!

Correctness

- **Mutual exclusion** is preserved
 - If process P_i is executing in its critical section, then no other processes can be executing in their critical sections
- The **Progress** requirement is satisfied
 - The selection (i.e., select a process that can enter the critical section) cannot be postponed indefinitely
- The **Bounded-Waiting** requirement is met
 - Time is bound after a process had made a request to enter its critical section and before that request is granted.

若一个 Process 想進入 C.S., 則他必定得在有限的時間內進入

Homework

- test_and_set()
- Bounded-waiting

Atomic Variables

- Typically, the `compare_and_swap()` instruction is not used directly to provide mutual exclusion
- It is used as a basic building block for constructing other tools that solve the critical-section problem
- One such tool is an [atomic variable](#)

Atomic Variables

```
increment(&sequence);
```

```
void increment(atomic_int *v) {  
    int temp;  
  
    do {  
        temp = *v;  
    } while (temp != compare_and_swap(v, temp, temp+1));  
}
```

Atomic Variables

- Note that atomic variables do not entirely solve race conditions in all circumstances
 - Producer-Consumer
 - Assume `count` is an atomic variable
 - 1 producer and 2 consumers, and the buffer is currently empty

```
while (true) {  
    while (count == 0)  
        ; /* do nothing */  
  
    next_consumed = buffer[out];  
    out = (out + 1)% BUFFER_SIZE;  
    count--;  
}
```

Consumer
Process

Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- (Spin Lock) Mutex Locks 互斥鎖
- Semaphores
- Monitors
- Liveness
- Summary

高階語言中，
實現 C.S. 的方法

} 兩個方法差在
「卡住其他 Process 的方法不同」

1. Mutex Lock: 用个 while 卡住

2. Semaphores: 利用 OS 的特性
(Chapter. 3. Process State Diagram)
將 Process 打進 waiting

Mutex Locks

互斥鎖 (Spin Lock)

- Previous solutions are generally **inaccessible** to application programmers
- OS designers build **higher-level** software tools to solve the critical-section problem
- The simplest tool is the **mutex lock** (**m**utual **e**xclusion)
- Two functions
 - The **acquire()** function acquires the lock
 - The **release()** function releases the lock

```
while (true) {  
    acquire lock  
    /* critical section */  
    release lock  
    /* remainder section */  
}
```

Mutex Locks

- Mutex locks can be implemented using the compare_and_swap() instruction
- The definitions of acquire() and release():

```
acquire() {  
    while (!available)  
        ; /* busy wait */  
    available = false  
}  
  
release() {  
    available = true;  
}
```

Mutex Locks

- Disadvantage
 - Busy waiting (**Spinlock**)
 - Another solution: **sleep**
- CPU「忙碌」的執行 while True 去檢查……
⇒ 浪費CPU資源

Process 1 Process 2 Process 1 Process 2



Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Semaphores

- A semaphore S is an integer variable
- Two atomic operations
 - `wait()` and `signal()`
- **Definition** (not Implementation)

```
wait(S) {  
    while (S <= 0)  
        ; // busy wait  
    S--;  
}
```

```
signal(S) {  
    S++;  
}
```

C++ 中, 还分成

- counting-semaphores
- binary-semaphores

Comparison

- https://medium.com/@chinhung_liu/work-note-semaphore-mutex-spinlock-391bba4c161a

Semaphore Usage

- Counting semaphore
 - The value of S can range over an unrestricted domain
 - Used to control access to a given resource consisting of a finite number of instances
 - Initialized to the number of resources available
- Binary semaphore
 - The value of S can range only between 0 and 1
 - Providing **mutual exclusion**

Semaphore Usage

```
wait(S) {                                signal(S) {
    while (S <= 0)                        S++;
        ; // busy wait                    }
    S--;
}
```

- To solve the critical-section problem
- Create a semaphore "mutex" initialized to 1

一不小心设成0?

会考!!!

```
while (true) {

    wait(mutex)

    /* critical section */

    signal(mutex)

    /* remainder section */
}
```

```
while (true) {

    wait(mutex)

    /* critical section */

    signal(mutex)

    /* remainder section */
}
```

Semaphore Usage

利用 Semaphore 控制不同 Process 之間
執行的順序

会考

- Execution Order
 - Require that S2 (Process 2) be executed only after S1 (Process 1) has completed
 - Create a semaphore "synch" initialized to 0

Process 1:

```
S1;  
signal(synch);
```

Process2:

```
wait(synch);  
S2;
```

Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Monitors

用 High-level 的角度
實現 C.S.

- When programmers use semaphores incorrectly
 - Situation 1
 - Interchanges the order
 - Several processes may be executing in their critical sections simultaneously

```
signal(mutex);
```

```
/* critical section */
```

```
wait(mutex);
```

Monitors

- When programmers use semaphores incorrectly
 - Situation 2
 - Replaces `signal(mutex)` with `wait(mutex)`
 - The process will permanently block on the second call to `wait()`, as the semaphore is now unavailable

```
wait(mutex);
```

```
/* critical section */
```

```
wait(mutex);
```

Monitors

Java 提供的 monitors: Synchronized

```
monitor monitor_name
{
    /* shared variable declarations */

    function P1 (...) {
        ...
    }

    function P2 (...) {
        ...
    }

    function Pn (...) {
        ...
    }

    initialization code (...) {
        ...
    }
}
```

Monitors

✦ AI Overview

In Java, the `synchronized` keyword is used to implement a monitor, which is a synchronization construct that provides mutual exclusion and cooperation among threads accessing shared resources.

SyncExample.java

```
synchronized void addX(int addValue) {  
    int localX = x;  
    localX += addValue;  
    long milliseconds = (long) (Math.random() * 3000);  
    System.out.println("addValue: " + addValue +  
        ", milliseconds: " + milliseconds);  
    try {  
        Thread.sleep(milliseconds);  
    } catch (InterruptedException e) {}  
    x = localX;  
}
```

Outline

- Race Condition
- Critical Section
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary

Liveness

- **Liveness** refers to a set or properties that a system must satisfy to ensure that process make progress during their execution life cycle
- A simple example of a liveness failure is an **infinite loop**
- **Deadlock** is also a situation that can lead to liveness failures

Spin-lock、Semaphores 雖能解決同步問題，
但若使用不當，可能會造成 Dead-Lock

Deadlock

P_0	P_1
<code>wait(S) ;</code>	<code>wait(Q) ;</code>
<code>wait(Q) ;</code>	<code>wait(S) ;</code>
<code>...</code>	<code>...</code>
<code>signal(S) ;</code>	<code>signal(Q) ;</code>
<code>signal(Q) ;</code>	<code>signal(S) ;</code>

Support that P_0 executes `wait(S)` and the P_1 executes `wait(Q)`

Outline

- Race Condition → 競爭條件, 2 Process 同時存取同一變數
- Critical Section → 解決 R.C. 的方法 — C.S.
- Peterson's Solution for the Critical Section Problem
- Hardware Support for Synchronization
- Mutex Locks
- Semaphores
- Monitors
- Liveness
- Summary



實現 C.S.
的方法

Summary

- Race Condition (3.1)
- Critical Section (3.2)
- Peterson's Solution (3.3)
- Synchronization tools ranged from low-level (e.g., `compare_and_swap()`) to increasingly higher-level (from `mutex locks` to `semaphores` to `monitors`) (3.4 ~ 3.7)
 - Busy waiting / Sleep
- Liveness (3.8)